

CLOUD & DEVOPS • KUBERNETES MANIFESTS

Kubernetes YAML

Reading and Writing the Files That Describe Your Cluster

01 YAML Syntax

02 Build a Pod, Piece by Piece

03 Apply It With kubectl

Vishwanath M S

Agenda

1

YAML Fundamentals

What YAML is, why Kubernetes uses it, and the two structures — Maps and Lists — that build everything else

2

Quick Review

The six recurring patterns you'll recognize in every manifest you read from here on

3

Building a Pod, Piece by Piece

Start from a complete Pod manifest, then zoom into each field until the whole file is legible

4

Applying It With kubectl

Turn the YAML file into a running Pod, and check on it

What is YAML?

YAML

stands for “Yet Another Markup Language,” or “YAML Ain't Markup Language” — depending who you ask.

USING YAML FOR KUBERNETES DEFINITIONS GIVES YOU:



Convenience

You no longer have to type every parameter on the command line — the whole config lives in one file.



Maintenance

YAML files can be added to source control, so every change to your cluster's config is tracked and reviewable.



Flexibility

You can express far more complex, nested structures than you'd ever want to type as command-line flags.

Two Structures — That's It

There are only two structures you need to know about in YAML. Everything else is a combination of these two.



Maps

Groups of key : value pairs — like a labeled field on a form.

```
apiVersion: v1
kind: Pod
```



Lists

A sequence of items, each one marked with a leading dash.

```
args:
- sleep
- "3000"
```

That's it. You might see maps of lists, lists of maps, and so on — but it's always built from these two.

YAML Maps — Key : Value Pairs

```
---  
  
apiVersion: v1  
kind: Pod
```

THE PATTERN

key: value

A colon and a space separate the key from its value. Every map in Kubernetes YAML follows this exact pattern.

The document separator — used when multiple YAML structures are defined in one file.

2 keys, 2 values

apiVersion maps to v1, and kind maps to Pod — two pre-defined keys, each holding one value.

YAML Maps — Nesting a Map Inside a Map

```
---  
  
apiVersion: v1  
kind: Pod  
metadata:  
  name: rss-site  
  labels:  
    app: web
```

A key can map to ANOTHER map, instead of a single value — that's how you build nested structure.



Indenting is important

The minimum step is 1 space — but 2 is the common convention, as shown here.



Indenting must be CONSISTENT

Every key at the same nesting level needs the exact same indentation.



NEVER use tabs in a YAML file — spaces only.

YAML Lists — A Sequence of Items

YAML lists are literally a sequence of objects — each one introduced by a dash and a space.

```
args:
```

- sleep
- "3000"
- message
- "Kube is Beautiful"

ITEMS IN THE LIST

Each - value line is one entry in the args list — four dashes, four items, in order.

Order matters — this list, if passed as a container's args, would run: sleep 3000 (with "message" and "Kube is Beautiful" as further args)

Maps Inside a List

Here's the pattern you'll see constantly in Kubernetes manifests: a list whose items are themselves maps.

```
containers:  
  - name: front-end  
    image: nginx  
    ports:  
      - containerPort: 80
```

- name: front-end

The dash starts a new item in the containers list — and that item is itself a map (name, image, ports).

- containerPort: 80

ports is itself a list — and its one item is a map containing a single key:value pair.

Quick Review — Six Patterns, Two Building Blocks

Maps

Groups of name:value pairs

```
kind: Pod
```

Lists

Individual items in sequence

```
- sleep  
- "3000"
```

Maps of Maps

A map whose value is another map

```
metadata:  
  name: rss-site
```

Maps of Lists

A map whose value is a list

```
args:  
  - sleep
```

Lists of Lists

A list whose items are themselves lists

```
- - a  
  - b
```

Lists of Maps

A list whose items are maps

```
- name: front-end  
  image: nginx
```

NOW, THE REAL THING

Building a Pod Manifest, Piece by Piece

Same Maps and Lists you just learned — just assembled into something real.

Creating a Pod Using YAML

This is the complete manifest we're about to take apart, field by field. Don't worry about reading it all yet.

```
---
apiVersion: v1
kind: Pod
metadata:
  name: rss-site
  labels:
    app: web
spec:
  containers:
    - name: front-end
      image: nginx
      ports:
        - containerPort: 80
    - name: rss-reader
      image: nickchase/rss-php-nginx:v1
      ports:
        - containerPort: 88
```

apiVersion

✓ ---

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: rss-site
```

```
  labels:
```

```
    app: web
```

We start with this

Every Kubernetes manifest opens by declaring which version of the API it targets. Currently, most core objects use v1.

kind

✓ ---

✓ apiVersion: v1

kind: Pod

metadata:

name: rss-site

labels:

app: web

Object type to be created

Here, we're specifying a Pod.

We could also create...

Deployment — a set of replicated Pods

Job — a Pod that runs to completion

Service — a stable network endpoint

metadata: name & labels

✓ ---

✓ apiVersion: v1

✓ kind: Pod

metadata:

name: rss-site

labels:

app: web

name — identity of the Pod

How this Pod is referred to by kubectl and by other objects in the cluster.

labels — tags for Kubernetes

Any key:value pair you like. Used as a label selector for finding and filtering this Pod later — Services and Replication Controllers match on exactly this.

spec: containers

...metadata block above unchanged...

spec:

containers:

```
- name: front-end  
  image: nginx  
  ports:  
    - containerPort: 80  
- name: rss-reader # 2nd container, same shape
```

The actual objects that make up the Pod

spec is where the real workload lives. Its properties can include:

- containers — what actually runs
- storage volumes — data the containers can share

container: name & image

...metadata block above unchanged...

```
✓ spec:
✓ containers:
  - name: front-end
    image: nginx
  ports:
    - containerPort: 80
  - name: rss-reader # 2nd container, same shape
```

name — the container's identity

The name given to the container we're trying to create — front-end.

image — where to pull from

The name of the image to pull from Docker Hub or an internal registry — here, nginx.

ports: containerPort

...metadata block above unchanged...

```
✓ spec:
✓   containers:
✓     - name: front-end
✓       image: nginx
      ports:
        - containerPort: 80
      - name: rss-reader # 2nd container, same shape
```

The port exposed to reach the container

containerPort: 80 tells Kubernetes which port inside the container should be reachable — nginx listens on 80 by default.

kubectl Commands

The manifest is just a text file until you apply it. Two commands take you from file to running Pod.

1

Run the YAML file on the cluster

```
$ kubectl apply -f <yaml filename>
```

Sends the manifest to the API server, which creates the Pod (and validates the YAML along the way).

2

Check the Pod status

```
$ kubectl get pods
```

Lists Pods in the current namespace — look for STATUS: Running once the containers have started.

RECAP

You should now be able to...

- Recognize YAML's two building blocks — Maps and Lists — and how they combine
- Explain the indentation and tabs-vs-spaces rules that keep a YAML file valid
- Read every field of a real Pod manifest: apiVersion, kind, metadata, and spec.containers
- Explain what a Label is for, and why it matters for how Pods get found later
- Apply a manifest with `kubectl apply -f` and confirm it's running with `kubectl get pods`



Next: Write and apply your own Pod manifest